

THE BUILD

Lesson 6



Lesson Plan

1

Automatization - Why do we need it

2

Bundlers - Types and Features

3

Node.js -> npm -> package.json

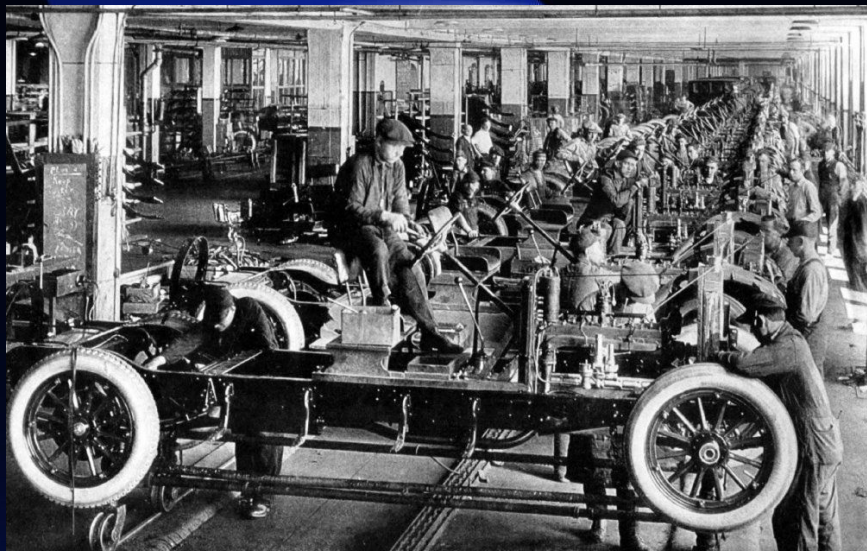
4

Vite

Automatization

is the process of transferring routine development tasks that can be done manually

- ✓ It **saves the developer's time**
- ✓ helps to **find and fix errors**
- ✓ generates code that **saves the browser's resources** and **speeds up website loading**.



BEFORE



AFTER

Problems

- **Code duplication makes maintenance difficult**
 - wrong copy & paste
 - missing some places
 - simple changes should be done repeatedly across entire project
- **Some files size may slow down a page loading**
 - photos
 - videos
 - scripts, styles

All item

Sort by ^

Search NFT

NFT x

ICO x

Crypto x

Clear all

Newest

Most popular

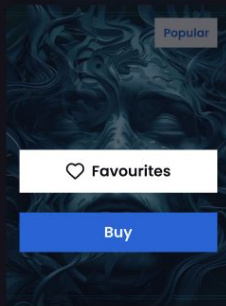
Increasing in price ✓

Lowering the price



\$199

Name Product



\$199

Name Product



\$199

Name Product



\$199

Name Product



\$199

Name Product



\$199

Name Product

Would be great if we can

- Extract repeated markup and styles into a separate components

```
{% macro card(title, text, tag) %}  
<div class="card">...</div>  
{% endmacro %}
```

```
.card {  
...  
}
```

- Use that component as some short name on the place

```
{% from "card.twig" import card %}  
{% card("$199", "Name Product", "New") %}
```

```
@use "card.scss"
```



\$199

Name Product

Popular builders

- ◆ **Gulp** – Task runner, can not built without plugins
- ◆ **Webpack** – The most popular and powerful, but complex
- ◆ **Parcel** – Simple, but slow
- ◆ **Vite** – Fast and convenient



We will use Vite



webpack



Vite



PARCEL

Node.js

– JavaScript Runtime
Outside the Browser



Why do we need it?



Allows running tools including our build



Manages dependencies in a project



Works with packages

NPM

Node Package Manager is a package manager for JavaScript.

📌 Main functions of npm:

- ✓ Installs and manages libraries and tools
- ✓ Works through the command line (terminal)
- ✓ Uses `package.json` to store project

dependencies

💡 npm comes bundled with Node.js – if you've installed Node.js, you already have npm!





Let's add the Build

Initialize

create new branch

```
cd ..
```

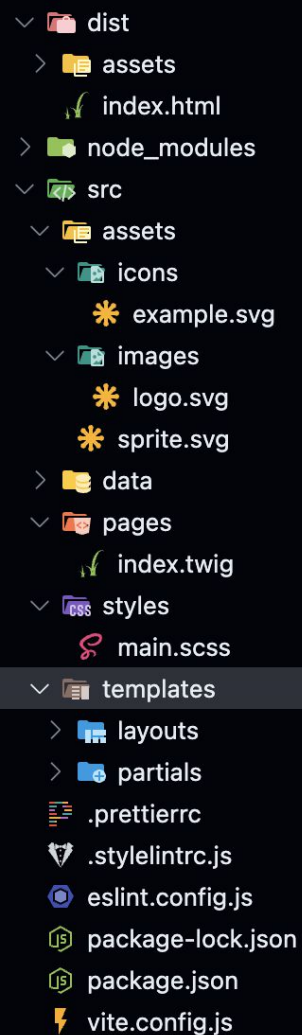
```
npx @bro-academy/create-vite-bundler binabox-username
```

your binabox project folder



Override? Yes.

Initialize a git repository? No.



- **dist/** — final build of the project
 - **assets/** — everything imported and not bundled into html
 - **index.html** — compiled HTML file
- **src/** — source files of the project
 - **assets/** — image storage
 - **icons/** — possibly SVG sprites
 - **images/** — regular images
 - **styles/** — source SCSS files
 - **pages/** — Twig page files
 - **templates/** — Twig template files
 - **layouts/** — base wrapper tags that we have on all pages
 - **partials/** — reusable components, e.g., `item-card.twig`

Configuration Files

- **.prettierrc**, **.stylelintrc.js**, **eslint.config.js** — configuration for formatting and linting code.
- **.gitignore** — files excluded from the repository.
- **vite.config.js** — Vite configuration file, responsible for automating the build process.
- **package.json**, **package-lock.json** — list of dependencies and scripts for working with the project.

.gitignore

.DS_Store

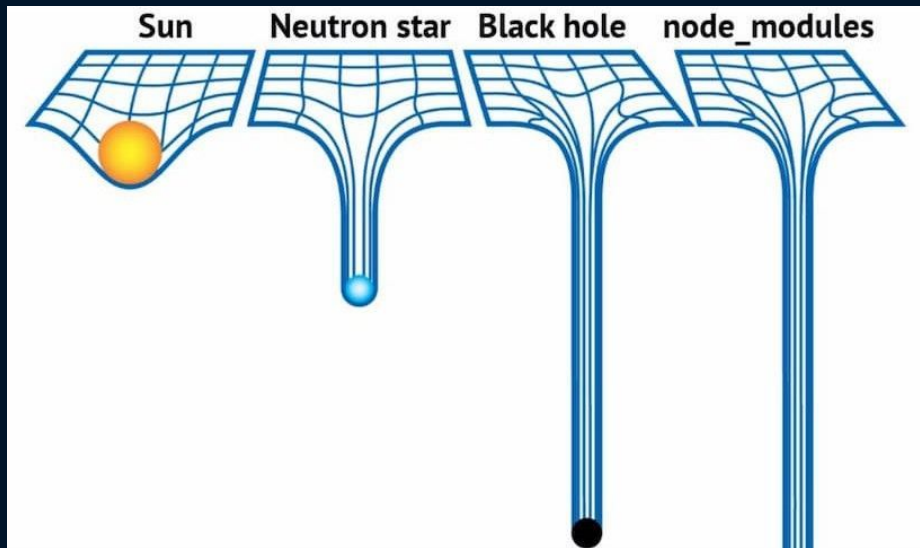
Thumbs.db

*.log

node_modules/ - too heavy

dist/ - version for prod

.gitignore from the build



Build commands for start

npm run build – will generate site (static build)

npm run dev – will generate site, watch files and **re-run build on change** (live server)

npm run preview - will just create a live server

ctrl + c – stop watch

package.json

↗ The `package.json` file stores project commands and dependencies.

Run in the Terminal:

▶ npm run `build`

```
{
  "name": "binabox",
  "version": "1.0.0",
  "scripts": {
    "dev": "vite dev",
    "build": "vite build",
    "preview": "vite preview",
  },
  "devDependencies": {
    "@vituum/vite-plugin-postcss": "^1.1.0",
    "@vituum/vite-plugin-twig": "^1.1.0",
    "sass-embedded": "^1.88.0",
    "vite": "^6.3.5",
    "vituum": "^1.2.0"
  }
}
```



Volta

Volta is a **JavaScript toolchain manager** that helps developers manage Node.js runtimes and package managers (like npm, Yarn, and pnpm) in a fast and consistent way.

We have pre-installed volta for you as a part of environment setup

When it is installed you may forget about it



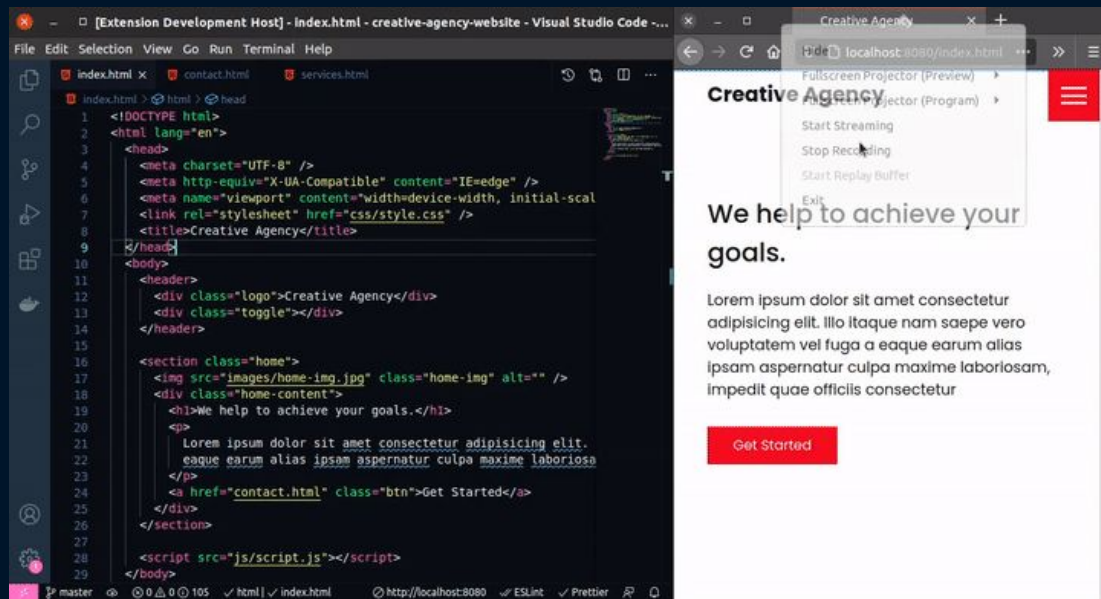
Live Server

Automatic Page Refresh

Edit →

Save →

Browser Updates!



CSS Processing

✓ Allows component approach

```
// SCSS                                     /* CSS */

$white: #fff;
$black: #000;

.button {
  text-align: center;
  color: $white;
  background-color: $black;
  border: 1px solid $black;
  padding: 1em 4em;
  &:hover {
    color: $black;
    background-color: $white;
    transition: 0.15s all linear;
  }
}

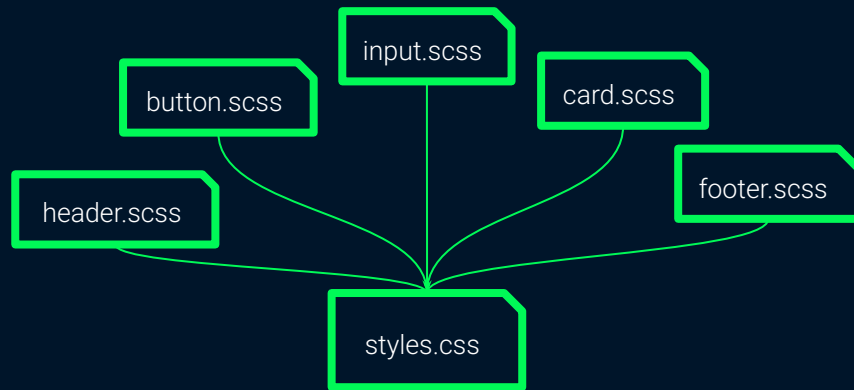
.button {
  text-align: center;
  color: #fff;
  background-color: #000;
  border: 1px solid #000;
  padding: 1em 4em;
}

.button:hover {
  color: #000;
  background-color: #fff;
  transition: 0.15s all linear;
}
```

✓ Make styles more cross browser

<pre>div { transition: all }</pre>	<pre>1 div { 2 -webkit-transition: all 1s ease-out; 3 -o-transition: all 1s ease-out; 4 transition: all 1s ease-out; 5 }</pre>
--------------------------------------	---

✓ Combines files



✓ Minifies code

```
.entry-content p {
  font-size: 14px !important;
}
```

```
.entry-content ul li {
  font-size: 14px !important;
}
```

```
.product_item p a {
  color: #000;
  padding: 10px 0px 0px 0;
  margin-bottom: 5px;
}
```

```
.entry-content p,.entry-content ul li
{font-size:14px!important}.product_item
p a{color:#000;padding:10px 0 0;margin-
bottom:5px;border-bottom:none}
```

Source maps

```
#topmenu #desktop-menu    _desktopmenu.scss:4
ul li {
  float: left;
  margin: 20px 20px 15px 20px;
  padding-bottom: 5px;
  line-height: 1;
  position: relative;
  cursor: pointer;
  text-align: center;
}
```

Sources

☐ Enable JavaScript source maps

☒ Detect indentation

☒ Autocompletion

☒ Bracket matching

Show whitespace characters: None

☒ Display variable values inline while debugging

☐ Enable CSS source maps

☐ Auto-reload generated CSS

Default indentation: 2 spaces

☒ Search in content scripts

Image Optimization & Responsive Images

📌 Processes images for performance optimization:

💡 Why?

- ✓ Converts images to **different sizes for responsive design**.
- ✓ Optimizes file size without losing quality.
- ✓ Generates modern formats like **WebP and AVIF**.

📌 Combines multiple **SVG icons** into a single optimized sprite file:

- Creates **SVG sprites** to reduce HTTP requests.

💡 We will use it later

Before

Quality: great

78

PageSpeed Score

Original image

JPG

1.2 MB



Optimize

After

Quality: great

100

PageSpeed Score

Optimized image

WebP

0.3 MB

prettier

What does it do?

- Automatically **formats code** to follow a consistent style.
- Works with **CSS, SCSS, JavaScript, JSON, and more**.
- **Fixes indentation, line breaks, spacing, and quotes**.

Why use prettier?

- ✓ **Formats code automatically** so you don't have to.
- ✓ **Works with multiple languages** (JavaScript, CSS, JSON, etc.).
- ✓ **Prevents unnecessary code diffs** in Git (e.g., inconsistent spacing).

Your code

```
body {
  margin: 0;   padding: 0;
background: #fafafa;
}

h1 {
  color: #145cf2 ; font-weight : 300;
font-family: Roboto,   sans-serif;
}
```

Beautify

Preview

```
body {
  margin: 0;
  padding: 0;
  background: #fafafa;
}

h1 {
  color: #145cf2;
  font-weight: 300;
  font-family: Roboto, sans-serif;
}
```

Copy

stylelint, eslint + htmlhint

📌 Ensures consistent coding style 🎨

✓ Checks CSS/SCSS and HTML code for errors and enforces coding style rules.

✓ Prevents common mistakes (e.g., unknown properties, missing quotes).

✓ Ensures the use of BEM naming convention for class names.

♦ Keeps your styles clean and maintainable! 🚀

```
> npx stylelint '**/*.scss'
```

```
src/styles/main.scss
```

```
60:1  ✖ ".ololo--ssss__sss" selector should be written by the BEM naming methodology:
      https://en.bem.info/methodology/naming-convention/#alternative-naming-schemes
61:15 ✖ Unexpected named color "red"
```

selector-class-pattern

color-named

```
✖ 2 problems (2 errors, 0 warnings)
```


The Key Differences Between These Tools

Feature	stylelint	eslint	prettier
Controls formatting in the editor	Able to setup	HTML & JS	✅ Yes
Linting & error checking	✅ Yes	✅ Yes	❌ No
Automatic code formatting	Able to setup for css/scss	Able to setup	✅ Yes
Works with	CSS & SCSS	HTML & JS	JS, CSS, JSON, etc.
Example rules	Class naming, color rules		Line breaks, spacing

Build commands for start

`npm run lint` - checks your HTML for mistakes

`npm run lint:fix` - will fix most popular of them

`npm run stylelint` - checks your CSS/SCSS for mistakes

`npm run stylelint:fix` - will fix most popular of them

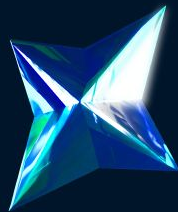
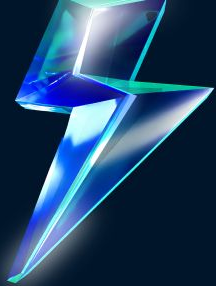
`npm run format` - will fix code style things

`npm run format:check` - will fix code style things

`npm run format:lint` - will fix code style things and then most popular mistakes

ctrl + c – stop watch

B Academy
RO

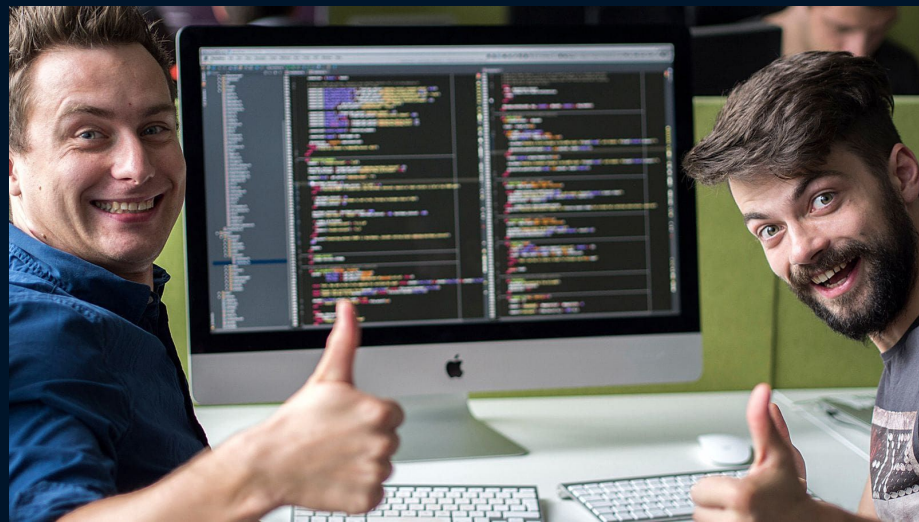


Break time



Summary – Now We Know:

- ✓ What a bundler is and why we need it
- ✓ What is Vite
- ✓ How to install and use them
- ✓ How to automate work with your project



Code easier, faster, and more efficiently 🚀

Quality Criteria for HTML Course

❤️ Mandatory for passing the course

🟡 Required for the highest grade

💚 Optional

❤️ The build is used

❤️ Folders `dist` and `node_modules` should not be in the GitHub repository

❤️ The project builds correctly

❤️ No red errors in the console during the build process

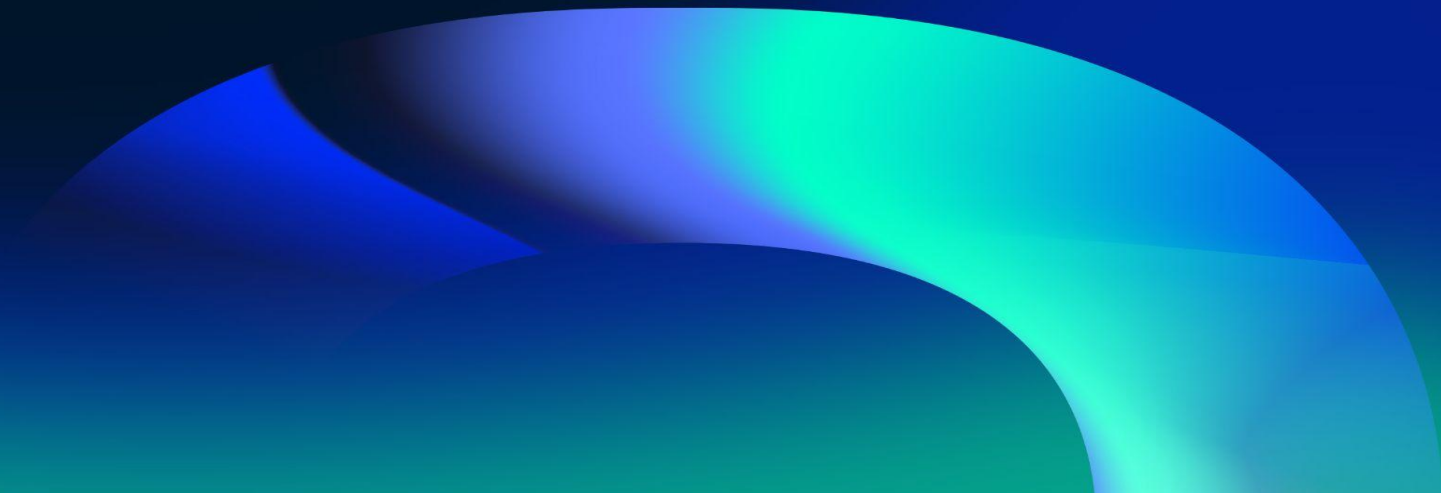
🟡 No warnings in the console during the build process

B Academy
RO



QUESTIONS?

Please fill out the feedback form
It's very important for us





THANK YOU!

Have a good evening!